



CATERPILLAR CAT®

Predictive Maintenance Platform

Technical Design & System Architecture Documentation

Status: Ready for Deploy

Version: 1.0.0 (Enterprise Spec)

Date: July 2026

Author: Caterpillar Hackathon Development Team

1. PROJECT OVERVIEW

The **Caterpillar CAT® Predictive Maintenance Platform** is an enterprise-grade Industrial Internet of Things (IIoT) analytics and fleet monitoring system. It provides site supervisors, maintenance engineers, service teams, and executives with real-time insight into the health, status, and telemetry profile of heavy machinery across globally distributed sites.

1.1 Business Problem Solved

Heavy machinery downtime represents one of the single largest losses in the mining, construction, and agricultural sectors. Unscheduled breakdowns result in direct revenue losses when processes halt, safety hazards to machinery operators, high emergency transport and parts replacement costs, and shortened overall lifespans of million-dollar equipment assets due to compounding faults.

1.2 Objective & Importance

By utilizing real-time sensor streams and automated machine learning, this platform transitions operations from reactive maintenance (fixing on failure) or scheduled maintenance (fixing on cycles regardless of wear) to data-driven predictive maintenance. This maximizes availability and saves hundreds of thousands of dollars in operational overhead.

1.3 Key Personas

- **Super Admin:** Fleet directors coordinating cross-facility health metrics.
- **Site Manager:** Local operations managers tracking active hardware status.
- **Maintenance Engineer:** Technicians resolving automated repair tickets.
- **Service Team Member:** Support staff logging parts replacements and oil levels.
- **Operator:** Machinery drivers receiving warning readouts on operational consoles.

2. TECHNOLOGY STACK

Technology	Responsibility	Key Advantage
Next.js (v16)	Frontend App Shell & Routing	Fast Edge pre-renders, robust static compilation
React	User Interface Elements	Stateful components, virtual DOM reconciliation
TypeScript	Type Safety	Compile-time validation, self-documenting code
Tailwind CSS	Enterprise Styling Theme	Utility classes, warm stone/yellow branding system
Django	REST API Backend & Core ORM	Secure authentication, simple migrations, relational logic
FastAPI	AI Evaluation Microservice	Asynchronous Python, lightweight memory usage
Neon Postgres	SQL Database Layer	Serverless auto-scaling, instant schema branching
SimpleJWT	User Authentication	Stateless JSON Web Tokens for API authorization

3. SYSTEM ARCHITECTURE

The platform relies on a decoupled microservices setup designed for high-availability. High-frequency sensor writes are fully separated from user interaction requests. Below is the data communication flow:

1. **Browser Client** → Queries Next.js UI.
 2. **Django API** → Handles JWT authentication, site definitions, and maintenance scheduling.
 3. **FastAPI Engine** → Performs Z-score anomaly sweeps and models failure probabilities.
 4. **Neon DB** → Serves as the central repository with time-series partitioned telemetry tables.

4. PROJECT FOLDER STRUCTURE

Folder	Contents & Purpose
/frontend	Next.js App Router, components/layout navigators, UI primitives (Stone & Yellow themes).
/backend	Django REST Framework Web Core. Includes apps users, machinery, telemetry, maintenance.
/ai-service	FastAPI microservice executing numpy risk coefficients computations and predictions writes.
/simulator	Python CLI daemon continuously feeding 105 machines' active metrics to PostgreSQL.

5. KEY MODULES & DATABASE SCHEMA

The system database schema is normalized to 3NF and optimized using PostgreSQL partitioning. Key tables include:

- **users**: Stores credentials and role IDs using cryptographically hashed values.
- **machines**: Relational asset profile tracking statuses (operational, warning, critical).
- **sensor_data**: Large-scale table partitioned by timestamp, saving temperature, vibration, and pressure logs.
- **predictions**: AI prediction registry storing calculated anomalies and RUL bounds.
- **maintenance_history**: Work order registry mapping task descriptions, technicians, costs, and statuses.

6. MAINTENANCE RESOLUTION WORKFLOW

The maintenance lifecycle provides automated coordination between sensor anomalies and active work orders:

1. **Anomaly Ingestion**: Sensor Simulator writes telemetry batches to Neon Postgres.
2. **AI Evaluation**: FastAPI identifies a sensor outlier (Z-Score > 2.5) and writes a prediction.
3. **Alert Triggered**: Django captures the prediction and issues an alarm notification.
4. **Task Scheduled**: An active ticket is generated in the *maintenance_history* registry.
5. **Work Order Complete**: The technician views the 'In Progress' tab, completes repairs, and updates the task state to completed. The machine's status returns to operational and the card moves to 'Complete'.

7. THE MAINTENANCE INTERACTION PORTAL

The bottom row of the Super Admin Dashboard features a single, optimized **MAINTENANCE** tabbed card. This widget incorporates:

- **Complete / In Progress Tabs:** Allows instant navigation between work lists without site refreshes.
- **Search by Machine Code:** React-filtered search checking machine signatures (e.g. CAT797F) ignoring whitespace and casing.
- **Cost Sort Selection:** Dynamically sorts list items numerically by cost in Low to High or High to Low order.
- **Date Sort Selection:** Reorders records by recently updated date metrics.

8. FUTURE ENHANCEMENTS

- **Apache Kafka Ingestion:** Route sensor messages to Kafka queues to scale telemetry handling.
- **Redis Caching:** Cache calculated machine health indicators to handle massive concurrent REST hits.
- **IoT Hub Edge Calculators:** Evaluate simple anomalies on local machine microcontrollers.